Non synchronized to synchronized list :

* **Collections** class has provided the following methods to convert non-synchronized collection into synchronized collection.

public static <T> List<T> synchronizedList(List<T> list) : non synchronized list to synchronized list

public static <T> Set<T> synchronizedSet(Set<T> set) : non synchronized set to synchronized set

public static <T> Map<K,V> synchronizedMap(Map<K,V> map) : non synchronized map to synchronized map

```
package com.ravi.arraylist;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.ListIterator;

//synchronizedList() + Reading in forward & Backward

public class ArrayListDemo5
{
   public static void main(String[] args)
   {
      ArrayList<String> cities = new ArrayList<>();
      cities.add("Hyderabad");
      cities.add("Delhi");
      cities.add("Banglore");
      cities.add("Chennai");

       List<String> synchronizedList = Collections.synchronizedList(cities);
       synchronizedList.forEach(IO::println);

       IO.println("Reading the data in forward & backward direction ");


       ListIterator<String> listIterator = synchronizedList.listIterator();

       IO.println("In Forward direction...");

       while(listIterator.hasNext())
       {
          IO.println(listIterator.next());
       }

       IO.println("In Backward direction...");


       while(listIterator.hasPrevious())
       {
          IO.println(listIterator.previous());
       }

   }
}
```

---

public native static long currentTimeMillis() :

* It is a predefined static method of **System class.**

* It is used to provide **Current System time in millisecond.**

* It is useful for comparison of two Java code.

Example :

`long currentSystemTime = System.currentTimeMillis();`

Note : currentSystemTime will provide current system time

```
package com.ravi.arraylist;

import java.util.ArrayList;
import java.util.Vector;

//Performance Comparison between ArrayList & Vector

public class ArrayListDemo6
{
     public static void main(String[] args)
     {
          final int ITERATION = 100000;

          Vector<Integer> vector = new Vector<>();

          long startTime = System.currentTimeMillis(); //2ms

          for(int i=0; i<=ITERATION; i++)
          {
               vector.add(i);
          }

          long endTime = System.currentTimeMillis(); //9ms

          IO.println("Time taken by Vector "+(endTime - startTime)+" ms");

          IO.println("..........................................");


      ArrayList<Integer> list = new ArrayList<>();

          startTime = System.currentTimeMillis(); //11ms

          for(int i=0; i<=ITERATION; i++)
          {
               list.add(i);
          }

          endTime = System.currentTimeMillis(); //18ms

          IO.println("Time taken by ArrayList "+(endTime - startTime)+" ms");

     }
}
```

Method return type can be generic :

```
void main()
{
 Lion lion = getLions().get(1); //No type casting is reqd
 IO.println(lion.name());
}

record Lion(String name)
{
}

public List<Lion> getLions()  //generic return type
{
  List<Lion> lions = new ArrayList<>();
  lions.add(new Lion("A"));
  lions.add(new Lion("B"));
  lions.add(new Lion("C"));
  return lions;
}
```

```
void main()
{
 Lion lion = (Lion) getLions().get(1); //Type casting is reqd because
 IO.println(lion.name());            //getLions() method return type is non-generic
}

record Lion(String name)
{
}

public List getLions()   //Method return type non-generic
{
  List<Lion> lions = new ArrayList<>();
  lions.add(new Lion("A"));
  lions.add(new Lion("B"));
  lions.add(new Lion("C"));
  return lions;
}
```

```
package com.ravi.arraylist;

import java.util.ArrayList;
import java.util.List;

//Get professors list based on the department

record Professor(String name, String specialization)
{
}

class Department
{
   private String name;
   private List<Professor> listOfProfessors;

   public Department(String name)
   {
    super();
    this.name = name;
    this.listOfProfessors = new ArrayList<>(); //composition
   }

   public String getName()
   {
       return this.name;
   }



   public void addProfessor(Professor professor)
   {
       listOfProfessors.add(professor);
   }

   public List<Professor> getListOfProfessors()
   {
       return this.listOfProfessors;
   }

}

public class ArrayListDemo7
{
    public static void main(String[] args)
    {
      Department cs = new Department("Computer Science Engineering");
      cs.addProfessor(new Professor("Mr Scott", "Java"));
      cs.addProfessor(new Professor("Mr Alen", "React"));
      cs.addProfessor(new Professor("Mr Smith", "Spring"));
      cs.addProfessor(new Professor("Mr John", "Spring AI/ML"));

      List<Professor> listOfProfessors = cs.getListOfProfessors();
      IO.println("All the professors in "+cs.getName()+" department");
      listOfProfessors.forEach(IO::println);

      IO.println("................................");

      Department civil = new Department("Civil Engineering");
      civil.addProfessor(new Professor("Mr Raj", "Basic Language"));
      civil.addProfessor(new Professor("Miss Priya", "Engineering Drawing"));
      civil.addProfessor(new Professor("Mr Smith", "C language"));
      civil.addProfessor(new Professor("Mr John", "Communication"));

      listOfProfessors = civil.getListOfProfessors();
      IO.println("All the professors in "+civil.getName()+" department");
      listOfProfessors.forEach(IO::println);

    }
}
```

```
package com.ravi.arraylist;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;
import java.util.Vector;

public class ArrayListDemo8
{
    @SuppressWarnings("unchecked")
    public static void main(String[] args) throws Exception
    {
        ArrayList<String> original = new ArrayList<>();
        original.add("BCA");
        original.add("MCA");
        original.add("BBA");
        original.add("B Tech");

        //1) Create another object by using Loose Coupling
        Vector<String> looseCoupling = new Vector<>(original);
        IO.println("Using constructor (Loose Coupling) :");
        IO.println(looseCoupling);

        //2) By using clone() method
        @SuppressWarnings("unchecked")
       ArrayList<String> cloned =(ArrayList<String>) original.clone();
        IO.println("Cloned Data :");
        IO.println(cloned);


        //3 Serialization & De-serialization
        String filePath = "D:\\new\\MyCourses.txt";

        //Serialization
        var fout = new FileOutputStream(filePath);
        var oos = new ObjectOutputStream(fout);

        try(fout ; oos)
        {
           oos.writeObject(original);
           IO.println("Object Stored...");
        }
        catch(Exception e)
        {
           e.printStackTrace();
        }

        //De-serialization
        var fin = new FileInputStream(filePath);
        var ois = new ObjectInputStream(fin);

        try(fin ; ois)
        {
           ArrayList<String> obj = (ArrayList<String>) ois.readObject();
           IO.println("ArrayList object using de-serialization :");
           IO.println(obj);
        }
        catch(Exception e)
        {
           e.printStackTrace();
        }
    }
}
```

---

public List<E> subList(int fromIndex, int toIndex) :

It is used to fetch/retrieve the part of the List based on the given index available in List interafce. The return type of this method is List, Here fromIndex is inclusive and toIndex is exclusive.

```
package com.ravi.arraylist;


import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;

public class ArrayListDemo9
{
     public static void main(String[] args)
     {
         ArrayList<Integer> list = new ArrayList<>();

      list.add(1); //0
      list.add(2); //1
      list.add(3);
      list.add(4);
      list.add(5);
      list.add(6);
      list.add(7);

      List<Integer> subList = list.subList(2,5); //3 to 5
      IO.println(subList);

     }
}
```

---

public void trimToSize() : [Reduce the capacity to current size]

Used to reduce the capacity equal to the current size.
After trim operation It is dynamically Growable.

public void ensureCapacity(int minCapacaity): [To avoid frequent resizing]

Increase the capacity of the ArrayList to avoid frequent resizing.

The minCapacaity parameter will specify that ArrayList will definetly hold the number of elements specified in the parameter of ensureCapacity() method.

```
package com.ravi.arraylist;


import java.util.ArrayList;
import java.util.HashSet;
import java.util.Set;

public class ArrayListDemo10 {

     public static void main(String[] args)
     {
          ArrayList<String> list = new ArrayList<>(100);
      list.add("Java");
      list.add("World");
      list.trimToSize();
      IO.println("Trimmed List Size: " + list.size());

      IO.println("........................");


      ArrayList<Integer> listOfNumber = new ArrayList<>();
      listOfNumber.add(999);

      listOfNumber.ensureCapacity(100);

      for (int i = 0; i < 50; i++)
      {
          listOfNumber.add(i);
      }

      IO.println("List size: " + listOfNumber.size());

     }

}
```

```
package com.ravi.arraylist;

import java.util.ArrayList;
import java.util.Vector;

public class ArrayListDemo11 {

     public static void main(String[] args)
     {
          Vector<Integer> v1 = new Vector<>();
          v1.add(100);
          v1.add(200);
          v1.add(300);

          ArrayList<Integer> al = new ArrayList<>();
          al.add(100);
          al.add(200);
          al.add(300);
          IO.println(al.equals(v1));
     }

}
```

```
public boolean equals(Object obj)
{
  if(obj instanceof List list)
  {
  }


}
```

Time complexity of ArrayList :

`ArrayList< Object> al = new ArrayList<>();`

`al.add(0, "Java");`

The time complexity of ArrayList to insert OR delete an element in the middle OR Begning (not at last) would be O(n) [Big O of n] because 'n' number of elements will be re-located so, it is not a good choice to perform insertion and deletion operation in the middle OR begning of the ArrayList.

On the other hand time complexity of ArrayList to retrieve an element from the List would be O(1) because by using get(int index) method we can retrieve the element randomly from the list. ArrayList class implements RandomAccess marker interface which provides the facility to fetch the elements Randomly.

In order to insert and delete the element in middle Or anywhere in the list frequently, we introduced LinkedList class.